

TechDesk - Modulos 8 a 14

Contents

TechDesk — Módulos 8 a 14 (Reescrito com Base no Projeto Real)	1
Continuação a partir do seu código atual, para você estudar e implementar sozinho . . .	1
Módulo 8 — Testes Automatizados (JUnit + Mockito)	2
Módulo 9 — Segundo Microsserviço: techdesk-notificacao-api	12
Módulo 10 — Comunicação Síncrona entre os Dois Serviços (REST)	15
Módulo 11 — Comunicação Assíncrona e Desacoplada (RabbitMQ)	17
Módulo 12 — Frontend Consumindo a API	21
Módulo 13 — Docker Compose Multi-Serviço (Aprofundamento)	23
Módulo 14 — Publicação, GitHub e Portfólio	25

TechDesk — Módulos 8 a 14 (Reescrito com Base no Projeto Real)

Continuação a partir do seu código atual, para você estudar e implementar sozinho

Este material substitui os Módulos 8 a 14 do curso original. Ele foi reescrito depois de uma leitura completa do seu projeto real (techdesk-chamados-api), então todo caminho de pacote, endpoint, nome de campo e assinatura de método aqui **é exatamente o que já existe no seu código** — não a versão genérica do curso anterior. Nada na sua pasta de projeto foi alterado: este documento é só para leitura e estudo; a implementação é sua.

O que seu projeto já tem (e este material assume como ponto de partida)

Seu projeto evoluiu além do curso original em vários pontos, e isso é bom — mostra iniciativa. O material abaixo já considera:

- Pacotes organizados **por funcionalidade** (chamados, categorias, tecnicos, usuarios, security, exception), não por camada. É uma escolha válida e muito usada em projetos reais — vamos seguir esse padrão em tudo que for novo.
- Endpoints no singular: /api/chamado e /api/categoria (não /api/chamados//api/categorias).
- Login com campos login e senha (não email).
- CRUD de Chamado com extras: criação em lote (POST /api/chamado/lote), exclusão em lote (DELETE /api/chamado), atualização parcial via PATCH com ChamadoPatchRequestDto.
- CRUD de Categoria com regra de negócio própria: não é possível excluir uma categoria que tem chamados associados (CategoriaPossuiChamadosException).
- Segurança com Spring Security + JWT (biblioteca com.auth0:java-jwt), papéis ADMIN/USER, e a entidade Usuario implementando UserDetails diretamente.
- GlobalExceptionHandler misturando ResponseEntity<ErroResponseDTO> (para validação e erro genérico) com ProblemDetail (para as exceções de negócio). Vamos explicar as duas abordagens na teoria do Módulo 8, porque isso é relevante para os testes.

- Spring Boot 4.1, o que muda algumas coisas de bastidor em relação a versões mais antigas (explicado logo abaixo) — inclusive **o que você já tem no pom.xml já é suficiente para o Módulo 8**, sem precisar adicionar nada novo.

Nota rápida sobre Spring Boot 4 e testes (para não se assustar com o que já está no seu pom.xml)

Seu pom.xml tem spring-boot-starter-webmvc-test e spring-boot-starter-validation-test, não o antigo spring-boot-starter-test. Isso é o novo padrão do Spring Boot 4: cada starter de produção (spring-boot-starter-webmvc, spring-boot-starter-validation, etc.) ganhou um starter de teste irmão, e **cada um desses starters de teste já traz o spring-boot-starter-test por baixo, de forma transitiva** — ou seja, JUnit, Mockito e AssertJ já estão disponíveis no seu projeto agora, sem precisar adicionar mais nada. Isso também significa que, embora o código de teste pareça idêntico ao que você veria num tutorial de Spring Boot 3, por baixo você está rodando **JUnit 6** (a versão atual, lançada em 2025) em vez do antigo JUnit 5 — na prática, para o que vamos fazer aqui, a API usada (org.junit.jupiter.api.*) é a mesma.

Módulo 8 — Testes Automatizados (JUnit + Mockito)

Objetivo

Escrever testes automatizados reais para ChamadoService, CategoriaService, TokenService e AuthService — substituindo definitivamente o hábito de testar “no olho” pelo Postman.

Teoria: como o JUnit funciona por dentro

O JUnit não é uma biblioteca só de anotações — ele é dividido em camadas:

- **JUnit Platform:** a camada de baixo nível que descobre e executa testes. É o “motor”. Qualquer ferramenta (Maven, sua IDE) fala com essa camada para saber quais testes existem e rodar cada um.
- **JUnit Jupiter:** o modelo de programação que você usa — as anotações @Test, @BeforeEach, @DisplayName, e as classes de assertiva (Assertions.assertEquals, assertThrows, etc). É a API que você importa (org.junit.jupiter.api.*).
- **Extensões:** o mecanismo pelo qual ferramentas externas se “plugam” no ciclo de vida do teste. É assim que o Mockito e o Spring participam dos seus testes — via @ExtendWith(MockitoExtension.class) ou @ExtendWith(SpringExtension.class) (este último ativado automaticamente por @SpringBootTest).

Por padrão, o JUnit cria **uma instância nova da classe de teste para cada método @Test** — é por isso que um teste nunca deveria depender do estado deixado por outro. Se dois testes precisam de dados diferentes, é sinal de que cada um deve montar seu próprio cenário do zero (padrão **Arrange-Act-Assert**: primeiro você monta o cenário, depois executa a ação, depois verifica o resultado).

Teoria: como o Mockito funciona por dentro

Quando você escreve @Mock ChamadoRepository chamadoRepository, o Mockito não usa uma implementação real do ChamadoRepository — ele **gera uma classe nova em tempo de execução** (usando manipulação de bytecode, via a biblioteca ByteBuddy) que implementa a mesma interface, mas cujos métodos, por padrão, não fazem nada e retornam valores “vazios” (null, 0, listas vazias). Esse objeto gerado é o **mock** — um dublê de teste.

A partir daí, duas operações são possíveis sobre esse dublê:

- **Stubbing** (`when(...).thenReturn(...)`): você programa o que aquele método deve devolver quando for chamado com determinados argumentos, sem executar nenhuma lógica real de banco de dados.
- **Verificação** (`verify(...)`): depois de rodar o código testado, você confirma que um determinado método do mock foi realmente chamado (e quantas vezes), o que é útil para garantir que uma ação (como salvar) de fato aconteceu.

`@InjectMocks` faz uma terceira coisa: cria uma instância **real** da classe que você quer testar (ex: `ChamadoService`) e injeta os mocks nela — via o construtor, se houver um compatível, ou via reflection nos campos. `@ExtendWith(MockitoExtension.class)` é quem inicializa tudo isso automaticamente antes de cada teste, sem você precisar chamar `MockitoAnnotations.openMocks(this)` manualmente.

Teoria: os três níveis de teste que vamos usar

1. **Teste unitário** (a maior parte deste módulo): testa uma classe isolada, com todas as dependências mockadas. Não sobe o Spring, não toca em banco de dados de verdade. Roda em milissegundos.
2. **Teste de integração com `@SpringBootTest`**: sobe o contexto real do Spring (todos os beans de verdade), incluindo a conexão com o banco. Mais lento, mas testa a integração real entre as camadas.
3. **MockMvc**: uma ferramenta que simula requisições HTTP **sem** precisar subir um servidor de verdade (Tomcat) numa porta de rede. Ele injeta a requisição diretamente no `DispatcherServlet` do Spring (o componente que roteia toda requisição HTTP para o Controller certo) e captura a resposta — por isso é rápido, mesmo testando o fluxo HTTP completo (Controller → Service → Repository).

8.1 — Testando o `ChamadoService` (unitário, com Mockito)

Crie o arquivo em: `techdesk-chamados-api/src/test/java/com/techdesk/techdesk/chamados/service/` (mesma estrutura de pacote do `ChamadoService` real, só que em `src/test` — é assim que se espelha a organização por funcionalidade também nos testes).

```
package com.techdesk.techdesk.chamados.service;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

import java.util.List;
import java.util.Optional;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import com.techdesk.techdesk.categorias.entity.Categoria;
import com.techdesk.techdesk.categorias.exception.CategoriaNaoEncontradaException;
import com.techdesk.techdesk.categorias.repository.CategoriaRepository;
import com.techdesk.techdesk.chamados.dto.ChamadoPatchRequestDto;
import com.techdesk.techdesk.chamados.dto.ChamadoRequestDTO;
import com.techdesk.techdesk.chamados.dto.ChamadoResponseDTO;
import com.techdesk.techdesk.chamados.entity.Chamado;
import com.techdesk.techdesk.chamados.entity.StatusChamado;
import com.techdesk.techdesk.chamados.exception.ChamadoNaoEncontradoException;
import com.techdesk.techdesk.chamados.exception.StatusChamadoNaoExisteException;
```

```

import com.techdesk.techdesk.chamados.repository.ChamadoRepository;

@ExtendWith(MockitoExtension.class)
class ChamadoServiceTest {

    @Mock
    private ChamadoRepository chamadoRepository; // dublê – não toca no banco de verdade

    @Mock
    private CategoriaRepository categoriaRepository;

    @InjectMocks
    private ChamadoService chamadoService; // instância real, com os dois mocks acima
    ↪ injetados no construtor

    @Test
    void deveCriarChamadoComSucesso() throws Throwable {
        // Arrange: monta o cenário
        Categoria categoria = new Categoria();
        categoria.setId(1L);
        categoria.setNome("Hardware");

        ChamadoRequestDTO dto = new ChamadoRequestDTO("Impressora não liga",
    ↪ "Detalhes...", 1L);

        when(categoriaRepository.findById(1L)).thenReturn(Optional.of(categoria));
        // thenAnswer devolve o próprio objeto passado para save(), simulando o
        ↪ comportamento real do JPA
        when(chamadoRepository.save(any(Chamado.class))).thenAnswer(inv ->
        ↪ inv.getArgument(0));

        // Act: executa o método testado
        ChamadoResponseDTO resultado = chamadoService.criar(dto);

        // Assert: confirma o resultado esperado
        assertEquals("Impressora não liga", resultado.titulo());
        assertEquals(StatusChamado.ABERTO, resultado.status());
        assertEquals("Hardware", resultado.categoriaNome());
        verify(chamadoRepository, times(1)).save(any(Chamado.class)); // confirma que save
        ↪ foi chamado exatamente 1 vez
    }

    @Test
    void deveLancarExcecaoAoCriarComCategoriaInexistente() {
        ChamadoRequestDTO dto = new ChamadoRequestDTO("Título", "Descrição", 99L);
        when(categoriaRepository.findById(99L)).thenReturn(Optional.empty());

        assertThrows(CategoriaNaoEncontradaException.class, () ->
        ↪ chamadoService.criar(dto));
        // Como a categoria não existe, o chamado NUNCA deveria ser salvo – vale a pena
        ↪ confirmar isso também:
        verify(chamadoRepository, never()).save(any());
    }

    @Test
    void deveLancarExcecaoQuandoChamadoNaoExisteAoBuscar() {
        when(chamadoRepository.findById(99L)).thenReturn(Optional.empty());

        assertThrows(ChamadoNaoEncontradoException.class, () ->
        ↪ chamadoService.buscarPorId(99L));
    }
}

```

```

}

@Test
void deveDefinirDataFechamentoAoMudarStatusParaFechado() throws Throwable {
    Chamado chamado = new Chamado();
    chamado.setId(1L);
    chamado.setStatusChamado(StatusChamado.ABERTO);
    Categoria categoria = new Categoria();
    categoria.setNome("Rede");
    chamado.setCategoria(categoria);

    when(chamadoRepository.findById(1L)).thenReturn(Optional.of(chamado));
    when(chamadoRepository.save(any(Chamado.class))).thenAnswer(inv ->
        ↪ inv.getArgument(0));

    ChamadoResponseDTO resultado = chamadoService.atualizarStatus(1L,
    ↪ StatusChamado.FECHADO);

    assertEquals(StatusChamado.FECHADO, resultado.status());
    assertNotNull(chamado.getDataFechamento()); // regra de negócio: fechar precisa
        ↪ registrar a data
}

@Test
void deveLancarExcecaoAoAtualizarComStatusInvalido() {
    Chamado chamado = new Chamado();
    chamado.setId(1L);
    Categoria categoria = new Categoria();
    categoria.setNome("Software");
    chamado.setCategoria(categoria);

    when(chamadoRepository.findById(1L)).thenReturn(Optional.of(chamado));

    ChamadoPatchRequestDto patch = new ChamadoPatchRequestDto(null, null,
    ↪ "STATUS_QUE_NAO_EXISTE", null);

    assertThrows(StatusChamadoNaoExisteException.class, () ->
        ↪ chamadoService.atualizaChamado(1L, patch));
}

@Test
void deveExcluirQuandoChamadoExiste() throws Throwable {
    when(chamadoRepository.existsById(1L)).thenReturn(true);

    chamadoService.excluir(1L);

    verify(chamadoRepository, times(1)).deleteById(1L);
}

@Test
void deveLancarExcecaoAoExcluirChamadoInexistente() {
    when(chamadoRepository.existsById(99L)).thenReturn(false);

    assertThrows(ChamadoNaoEncontradoException.class, () ->
        ↪ chamadoService.excluir(99L));
    verify(chamadoRepository, never()).deleteById(any());
}

@Test

```

```

void deveCriarChamadosEmLote() {
    Categoria categoria = new Categoria();
    categoria.setId(1L);
    categoria.setNome("Hardware");

    List<ChamadoRequestDTO> dtos = List.of(
        new ChamadoRequestDTO("Chamado 1", "Desc 1", 1L),
        new ChamadoRequestDTO("Chamado 2", "Desc 2", 1L)
    );

    when(categoriaRepository.findById(1L)).thenReturn(Optional.of(categoria));
    when(chamadoRepository.saveAll(any())).thenAnswer(inv -> inv.getArgument(0));

    List<ChamadoResponseDTO> resultado = chamadoService.criarEmLote(dtos);

    assertEquals(2, resultado.size());
    assertEquals("Chamado 1", resultado.get(0).titulo());
}
}

```

Por que testar tanto o caminho de sucesso quanto o de erro? Um teste que só cobre “deu certo” não protege contra regressão nas regras de negócio. Repare que testamos explicitamente: sucesso, categoria inexistente, chamado inexistente, status inválido, e a regra de “fechar registra data de fechamento”. Isso é o que um recrutador técnico entende como “cobertura de teste séria” — não é sobre número de testes, é sobre quais decisões de negócio estão protegidas.

8.2 — Testando o CategoriaService (unitário)

Crie em: techdesk-chamados-api/src/test/java/com/techdesk/techdesk/categorias/service/CategoriaServiceTest

package com.techdesk.techdesk.categorias.service;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

import java.util.Optional;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import com.techdesk.techdesk.categorias.dto.CategoriaRequestDto;
import com.techdesk.techdesk.categorias.entity.Categoria;
import com.techdesk.techdesk.categorias.exception.CategoriaJaExisteException;
import com.techdesk.techdesk.categorias.exception.CategoriaNaoEncontradaException;
import com.techdesk.techdesk.categorias.exception.CategoriaPossuiChamadosException;
import com.techdesk.techdesk.categorias.repository.CategoriaRepository;
import com.techdesk.techdesk.chamados.repository.ChamadoRepository;

@ExtendWith(MockitoExtension.class)
class CategoriaServiceTest {

@Mock
private CategoriaRepository categoriaRepository;

@Mock

```

private ChamadoRepository chamadoRepository;

@InjectMocks
private CategoriaService categoriaService;
// Nota: o construtor real de CategoriaService recebe também a Application como
//      ↳ parâmetro
// (provavelmente um resquício de geração automática) – o Mockito ignora parâmetros que
// não tem um @Mock correspondente, então @InjectMocks funciona normalmente aqui.

@Test
void deveCriarCategoriaComSucesso() throws Throwable {
    CategoriaRequestDto dto = new CategoriaRequestDto("Hardware");
    when(categoriaRepository.findByNome("Hardware")).thenReturn(Optional.empty());
    when(categoriaRepository.save(any(Categoria.class))).thenAnswer(inv ->
        ↳ inv.getArgument(0));

    var resultado = categoriaService.criar(dto);

    assertEquals("Hardware", resultado.nome());
}

@Test
void deveLancarExcecaoAoCriarCategoriaDuplicada() {
    CategoriaRequestDto dto = new CategoriaRequestDto("Hardware");
    Categoria existente = new Categoria();
    existente.setNome("Hardware");
    when(categoriaRepository.findByNome("Hardware")).thenReturn(Optional.of
        ↳ (existente));

    assertThrows(CategoriaJaExisteException.class, () -> categoriaService.criar(dto));
}

@Test
void naoDeveExcluirCategoriaComChamadosAssociados() {
    when(categoriaRepository.existsById(1L)).thenReturn(true);
    when(chamadoRepository.existsByCategoriaId(1L)).thenReturn(true);

    assertThrows(CategoriaPossuiChamadosException.class, () ->
        ↳ categoriaService.excluirCategoria(1L));
    verify(categoriaRepository, never()).deleteById(any());
}

@Test
void deveExcluirCategoriaSemChamadosAssociados() {
    when(categoriaRepository.existsById(1L)).thenReturn(true);
    when(chamadoRepository.existsByCategoriaId(1L)).thenReturn(false);

    categoriaService.excluirCategoria(1L);

    verify(categoriaRepository, times(1)).deleteById(1L);
}

@Test
void deveLancarExcecaoAoExcluirCategoriaInexistente() {
    when(categoriaRepository.existsById(99L)).thenReturn(false);

    assertThrows(CategoriaNaoEncontradaException.class, () ->
        ↳ categoriaService.excluirCategoria(99L));
}

```

```
}
```

Repare que este teste força você a olhar de novo para o construtor de `CategoriaService`, que recebe um `TechdeskChamadosApiApplication` sem usar — isso é código morto (parâmetro nunca lido). Não é um problema que quebra nada, mas é o tipo de coisa que um teste bem escrito acaba revelando. Vale limpar quando for mexer nessa classe de novo.

8.3 — Testando o `TokenService` (unitário, sem Spring)

Este é o mais interessante tecnicamente, porque `TokenService` usa `@Value` para injetar `jwt.secret` e `jwt.expiration-minutes` — e num teste unitário puro (sem subir o Spring), essas anotações **não são processadas**, então os campos ficam `null/0`. Resolvemos isso com `ReflectionTestUtils`, uma ferramenta do próprio Spring Test para setar campos privados diretamente, sem precisar subir o contexto inteiro só para testar uma classe pequena.

Crie em: `techdesk-chamados-api/src/test/java/com/techdesk/techdesk/security/service/TokenServiceTest.java`

```
package com.techdesk.techdesk.security.service;

import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.test.util.ReflectionTestUtils;

import com.techdesk.techdesk.usuarios.entity.UserRole;
import com.techdesk.techdesk.usuarios.entity.Usuario;

class TokenServiceTest {

    private TokenService tokenService;

    @BeforeEach
    void setUp() {
        tokenService = new TokenService();
        // Sem o Spring rodando, @Value não é processado – setamos manualmente via
        // ↪ reflection.
        ReflectionTestUtils.setField(tokenService, "secret",
        ↪ "chave-de-teste-bem-longa-para-o-hmac256");
        ReflectionTestUtils.setField(tokenService, "expirationMinutes", 60L);
    }

    @Test
    void deveGerarTokenEValidarComSucesso() {
        Usuario usuario = new Usuario("elimar", "hashSenha", UserRole.USER);

        String token = tokenService.generateToken(usuario);

        assertNotNull(token);
        assertFalse(token.isBlank());

        String login = tokenService.validateToken(token);
        assertEquals("elimar", login); // o subject do token deve ser o login usado para
        ↪ gerá-lo
    }

    @Test
    void deveRetornarNuloParaTokenInvalido() {
```

```

        String resultado = tokenService.validateToken("token.completamente.invalido");
        assertNull(resultado);
    }

    @Test
    void deveRetornarNuloParaTokenExpirado() throws InterruptedException {
        ReflectionTestUtils.setField(tokenService, "expirationMinutes", 0L); // expira
↪ imediatamente
        Usuario usuario = new Usuario("elimar", "hashSenha", UserRole.USER);

        String token = tokenService.generateToken(usuario);
        Thread.sleep(1000); // garante que o relógio avançou além da expiração

        assertNull(tokenService.validateToken(token));
    }
}

```

Atenção a um detalhe do seu application.yaml: jwt.expiration-minutes está configurado como 1 (um minuto). Isso é ótimo pra você enxergar expiração de token acontecendo de verdade, mas vai atrapalhar seus testes manuais no Postman — se demorar mais de 1 minuto entre o login e a próxima chamada, o token expira. Vale subir esse valor temporariamente (por exemplo, para 60) enquanto você desenvolve, e voltar a um valor baixo só quando for demonstrar a expiração de propósito.

8.4 — Testando o AuthService (unitário, com múltiplos mocks)

Crie em: techdesk-chamados-api/src/test/java/com/techdesk/techdesk/security/service/AuthServiceTest.java

```
package com.techdesk.techdesk.security.service;
```

```
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;
```

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
```

```
import com.techdesk.techdesk.usuarios.dto.LoginRequestDTO;
import com.techdesk.techdesk.usuarios.dto.RegisterRequest;
import com.techdesk.techdesk.usuarios.entity.UserRole;
import com.techdesk.techdesk.usuarios.entity.Usuario;
import com.techdesk.techdesk.usuarios.repository.UsuarioRepository;
```

```
@ExtendWith(MockitoExtension.class)
```

```
class AuthServiceTest {
```

```
    @Mock private AuthenticationManager authenticationManager;
    @Mock private UsuarioRepository userRepository;
    @Mock private TokenService jwtService;
    @Mock private PasswordEncoder passwordEncoder;
    @Mock private Authentication authentication; // o resultado que o
↪ AuthenticationManager devolve

```

```
    @InjectMocks

```

```

private AuthService authService;

@Test
void deveFazerLoginComSucesso() {
    Usuario usuario = new Usuario("elimar", "hashSenha", UserRole.USER);
    LoginRequestDTO request = new LoginRequestDTO("elimar", "senha123");

    // authenticate() dispara, por baixo, o UserDetailsServiceImpl + validação de
    //   ↳ senha via BCrypt —
    // aqui simulamos só o RESULTADO desse processo, sem executar a lógica real de
    //   ↳ novo.
    when(authenticationManager.authenticate(any())).thenReturn(authentication);
    when(authentication.getPrincipal()).thenReturn(usuario);
    when(jwtService.generateToken(usuario)).thenReturn("token-fake-de-teste");

    var resposta = authService.login(request);

    assertEquals("token-fake-de-teste", resposta.token());
    assertEquals("Bearer", resposta.TokenType());
}

@Test
void deveRegistrarNovoUsuarioComSucesso() {
    RegisterRequest request = new RegisterRequest("novoUsuario", "senha123");
    when(userRepository.existsByLogin("novoUsuario")).thenReturn(false);
    when(passwordEncoder.encode("senha123")).thenReturn("senhaCriptografada");

    var resposta = authService.register(request);

    assertEquals("novoUsuario", resposta.login());
    assertEquals(UserRole.USER, resposta.role()); // registro público sempre cria como
    //   ↳ USER, nunca ADMIN
    verify(userRepository, times(1)).save(any(Usuario.class));
}

@Test
void naoDeveRegistrarLoginJaExistente() {
    RegisterRequest request = new RegisterRequest("elimar", "senha123");
    when(userRepository.existsByLogin("elimar")).thenReturn(true);

    assertThrows(IllegalArgumentException.class, () -> authService.register(request));
    verify(userRepository, never()).save(any());
}
}

```

Ponto de atenção real, não só de teste: AuthService.register sempre cria o usuário com UserRole.USER — não existe hoje nenhum jeito de criar um ADMIN pela API (nem deveria existir um endpoint público para isso). Se você precisar de um usuário ADMIN para testar as rotas /admin/**, vai ter que inserir direto no banco (UPDATE usuarios SET role = 'ADMIN' WHERE login = '...') ou criar uma rotina interna separada. É um ponto interessante para comentar em entrevista: “temos uma rota administrativa protegida por role, mas a promoção a admin é deliberadamente manual, fora da API pública” — é uma decisão de segurança válida.

8.5 — Um teste de integração com MockMvc

Este é diferente dos anteriores: em vez de mockar tudo, ele sobe o contexto real do Spring (@SpringBootTest) e testa o comportamento HTTP de verdade — por isso precisa do MySQL do

Docker rodando (docker-compose up -d, na raiz techdesk/).

Crie em: techdesk-chamados-api/src/test/java/com/techdesk/techdesk/chamados/controller/Chamado

```
package com.techdesk.techdesk.chamados.controller;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.web.servlet.MockMvc;

@SpringBootTest
@AutoConfigureMockMvc
class ChamadoControllerIT {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void deveBloquearAcessoSemToken() throws Exception {
        mockMvc.perform(get("/api/chamado"))
            .andExpect(status().isUnauthorized()); // ou isForbidden(), dependendo da
    }
}
```

Se a linha do import `...AutoConfigureMockMvc` não resolver automaticamente com esse caminho na sua versão do Spring Boot 4, deixe a IDE corrigir o import (Alt+Enter no IntelliJ) — os pacotes de teste foram reorganizados nessa versão, e sua IDE resolve isso mais rápido do que qualquer lista fixa de imports.

Onde e como rodar os testes

No terminal, **dentro de techdesk/techdesk-chamados-api/**:

```
mvn test
```

Ou, na IDE: botão direito na pasta src/test/java → “Run All Tests”. O relatório de cada execução fica salvo em techdesk-chamados-api/target/surefire-reports/ — você já tem um exemplo desse relatório gerado lá (do teste contextLoads padrão), então já sabe onde procurar.

Checkpoint — o que você deve ter, e como confirmar

- mvn test termina com BUILD SUCCESS e a linha final mostra algo como Tests run: 20, Failures: 0, Errors: 0, Skipped: 0.
- Se um teste falhar, a mensagem do assertEquals/assertThrows diz exatamente o valor esperado vs. o valor recebido — leia essa mensagem antes de mexer em qualquer código.
- Você consegue explicar, sem consultar nada: a diferença entre @Mock e @InjectMocks, por que TokenServiceTest precisa de ReflectionTestUtils, e por que o teste de AuthService.register verifica verify(userRepository, never()).save(any()) no caso de login duplicado (não basta lançar a exceção — é preciso garantir que nada foi salvo antes dela).
- Faça commit: git add . && git commit -m "test: testes unitários de ChamadoService, CategoriaService, TokenService e AuthService".

Módulo 9 — Segundo Microserviço: techdesk-notificacao-api

Objetivo

Criar um segundo serviço, independente, que registra notificações relacionadas aos chamados — o primeiro passo para a arquitetura de microserviços que fecha seu portfólio.

Teoria: por que um segundo serviço, e não só uma classe nova

Um microserviço de verdade tem processo próprio, banco de dados próprio (ou schema próprio) e ciclo de deploy próprio. A regra mais importante: **cada serviço é dono dos seus próprios dados** — o techdesk-notificacao-api nunca vai acessar o banco do techdesk-chamados-api diretamente, só recebe informação via API (Módulo 10) ou evento (Módulo 11). Isso é o que permite os dois evoluírem, escalarem e até falharem de forma independente.

Pacotes do novo projeto (seguindo o mesmo padrão do chamados-api: por funcionalidade)

Sugestão de package base: `com.techdesk.notificacao` (paralelo ao `com.techdesk.techdesk` que você já usa, mas sem repetir “techdesk” duas vezes).

```
com.techdesk.notificacao
├── NotificacaoApiApplication.java
├── config/
│   └── RabbitMQConfig.java (Módulo 11)
├── notificacoes/
│   ├── controller/NotificacaoController.java
│   ├── dto/NotificacaoResponseDTO.java
│   ├── entity/Notificacao.java
│   ├── repository/NotificacaoRepository.java
│   ├── service/NotificacaoService.java
│   └── messaging/ (Módulo 11)
│       ├── ChamadoCriadoEvent.java
│       └── ChamadoEventListener.java
```

Passo a passo

1. Em [start.spring.io](#): artifactId: techdesk-notificacao-api, package `com.techdesk.notificacao`, Java 21, Spring Boot 4.x, dependências **só as necessárias agora**: Spring Web (que no Spring Boot 4 é `spring-boot-starter-webmvc`), Spring Data JPA, MySQL Driver, Validation, DevTools. Sem Security (esse serviço não tem login) e sem AMQP ainda (só entra no Módulo 11, mesma regra que já seguimos no chamados-api).
2. Extraia de forma que a pasta fique em `techdesk/techdesk-notificacao-api/` (irmã de `techdesk-chamados-api/`).
3. Adicione um terceiro banco ao `docker-compose.yml` da raiz. Seu chamados-api já usa a porta externa 3307 (mapeada para a 3306 interna do container) — para não colidir nem com ela nem com um MySQL local na porta padrão 3306, use 3308:

```
# trecho a adicionar em "services:" dentro de techdesk/docker-compose.yml
mysql-notificacao:
  image: mysql:8.0
  container_name: techdesk-mysql-notificacao
  environment:
    MYSQL_ROOT_PASSWORD: techdesk123
    MYSQL_DATABASE: techdesk_notificacao
  ports:
    - "3308:3306"
```

Aplique com docker-compose up -d (dentro de techdesk/) e confirme com docker ps — deve aparecer um terceiro container.

```
// techdesk-notificacao-
↳ api/src/main/java/com/techdesk/notificacao/notificacoes/entity/Notificacao.java
package com.techdesk.notificacao.notificacoes.entity;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "notificacoes")
public class Notificacao {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long chamadoId; // referência ao chamado de OUTRO serviço – guardamos só o id,
    ↳ nunca o objeto inteiro
    private String tituloChamado;
    private String mensagem;
    private LocalDateTime dataEnvio;

    public Notificacao() {}

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public Long getChamadoId() { return chamadoId; }
    public void setChamadoId(Long chamadoId) { this.chamadoId = chamadoId; }
    public String getTituloChamado() { return tituloChamado; }
    public void setTituloChamado(String tituloChamado) { this.tituloChamado =
    ↳ tituloChamado; }
    public String getMensagem() { return mensagem; }
    public void setMensagem(String mensagem) { this.mensagem = mensagem; }
    public LocalDateTime getDataEnvio() { return dataEnvio; }
    public void setDataEnvio(LocalDateTime dataEnvio) { this.dataEnvio = dataEnvio; }
}
```

```
// techdesk-notificacao-
↳ api/src/main/java/com/techdesk/notificacao/notificacoes/repository/NotificacaoRepository.java
package com.techdesk.notificacao.notificacoes.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import com.techdesk.notificacao.notificacoes.entity.Notificacao;

public interface NotificacaoRepository extends JpaRepository<Notificacao, Long> {
    List<Notificacao> findByChamadoId(Long chamadoId);
}
```

```
// techdesk-notificacao-
↳ api/src/main/java/com/techdesk/notificacao/notificacoes/service/NotificacaoService.java
package com.techdesk.notificacao.notificacoes.service;

import java.time.LocalDateTime;
import java.util.List;
import org.springframework.stereotype.Service;
import com.techdesk.notificacao.notificacoes.entity.Notificacao;
import com.techdesk.notificacao.notificacoes.repository.NotificacaoRepository;
```

```

@Service
public class NotificacaoService {

    private final NotificacaoRepository repository;

    public NotificacaoService(NotificacaoRepository repository) {
        this.repository = repository;
    }

    public Notificacao registrar(Long chamadoId, String tituloChamado, String mensagem) {
        Notificacao notificacao = new Notificacao();
        notificacao.setChamadoId(chamadoId);
        notificacao.setTituloChamado(tituloChamado);
        notificacao.setMensagem(mensagem);
        notificacao.setDataEnvio(LocalDateTime.now());
        return repository.save(notificacao);
    }

    public List<Notificacao> listarPorChamado(Long chamadoId) {
        return repository.findByChamadoId(chamadoId);
    }
}

```

```

// techdesk-notificacao-
↪ api/src/main/java/com/techdesk/notificacao/notificacoes/controller/NotificacaoController.java
package com.techdesk.notificacao.notificacoes.controller;

```

```

import java.util.List;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import com.techdesk.notificacao.notificacoes.entity.Notificacao;
import com.techdesk.notificacao.notificacoes.service.NotificacaoService;

```

```

@RestController
@RequestMapping("/api/notificacao")
public class NotificacaoController {

    private final NotificacaoService service;

    public NotificacaoController(NotificacaoService service) {
        this.service = service;
    }

    @GetMapping("/chamado/{chamadoId}")
    public ResponseEntity<List<Notificacao>> listarPorChamado(@PathVariable Long
        ↪ chamadoId) {
        return ResponseEntity.ok(service.listarPorChamado(chamadoId));
    }
}

```

```

# techdesk-notificacao-api/src/main/resources/application.yaml
spring:
  application:
    name: techdesk-notificacao-api
  datasource:
    url: jdbc:mysql://localhost:3308/techdesk_notificacao
    username: root
    password: techdesk123
  jpa:
    hibernate:

```

```
ddl-auto: update
show-sql: true
server:
port: 8081    # o chamados-api já usa 8080
```

Suba os dois serviços em terminais separados: **Terminal 1** (techdesk-chamados-api/) e **Terminal 2** (techdesk-notificacao-api/), ambos com `mvn spring-boot:run`.

Checkpoint — o que você deve ter, e como confirmar

- `docker ps` mostra 3 containers (techdesk-mysql, techdesk-rabbitmq, techdesk-mysql-notificacao).
- Os dois serviços sobem ao mesmo tempo, em portas diferentes (8080 e 8081), sem conflito.
- `GET http://localhost:8081/api/notificacao/chamado/1` responde 200 OK com uma lista vazia.
- Commit: `git add . && git commit -m "feat(notificacao-api): estrutura inicial do segundo microsserviço"`.

Módulo 10 — Comunicação Síncrona entre os Dois Serviços (REST)

Objetivo

Fazer o techdesk-chamados-api chamar o techdesk-notificacao-api via REST toda vez que um chamado é criado, e sentir na prática o problema de acoplamento que isso gera.

Teoria: RestClient e o problema do acoplamento síncrono

RestClient é o cliente HTTP moderno do Spring (substituiu o antigo RestTemplate como recomendação padrão) para chamar outra API a partir do seu backend. Ele já está disponível no seu projeto sem dependência nova, porque vem transitivamente junto com o `spring-boot-starter-webmvc` que você já tem.

O problema real que este módulo demonstra: se o chamados-api **espera a resposta** do notificacao-api antes de continuar, qualquer lentidão ou queda no serviço de notificação passa a afetar a operação principal (criar um chamado) — mesmo essa sendo, do ponto de vista do negócio, uma consequência secundária. É exatamente esse problema que o Módulo 11 resolve com mensageria.

```
// techdesk-chamados-api/src/main/java/com/techdesk/techdesk/config/RestClientConfig.java
package com.techdesk.techdesk.config;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestClient;
```

```
@Configuration
public class RestClientConfig {

    @Bean
    public RestClient notificacaoRestClient() {
        return RestClient.builder()
            .baseUrl("http://localhost:8081")
            .build();
    }
}
```

```
// techdesk-chamados-
↪ api/src/main/java/com/techdesk/techdesk/chamados/service/NotificacaoClient.java
package com.techdesk.techdesk.chamados.service;

import java.util.Map;
import org.springframework.http.MediaType;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;

// Classe dedicada a falar com o OUTRO serviço – mantém essa responsabilidade
// isolada, fora da regra de negócio de ChamadoService.
@Service
public class NotificacaoClient {

    private final RestClient restClient;

    public NotificacaoClient(RestClient notificacaoRestClient) {
        this.restClient = notificacaoRestClient;
    }

    public void notificar(Long chamadoId, String titulo) {
        restClient.post()
            .uri("/api/notificacao")
            .contentType(MediaType.APPLICATION_JSON)
            .body(Map.of(
                "chamadoId", chamadoId,
                "tituloChamado", titulo,
                "mensagem", "Novo chamado aberto: " + titulo
            ))
            .retrieve()
            .toBodilessEntity();
    }
}
}
```

Adicione também, no notificacao-api, um POST /api/notificacao no NotificacaoController (chamando service.registrar(...) com o corpo recebido — crie um DTO de requisição simples para isso).

No ChamadoService, injete NotificacaoClient no construtor e chame no final de criar(...):

```
public ChamadoResponseDTO criar(ChamadoRequestDTO dto) throws Throwable {
    // ... lógica existente ...
    Chamado salvo = chamadoRepository.save(chamado);

    notificacaoClient.notificar(salvo.getId(), salvo.getTitulo()); // chamada síncrona ao
↪ outro serviço

    return toResponseDTO(salvo);
}
```

Teste no Postman

1. Com os dois serviços rodando, crie um chamado (POST http://localhost:8080/api/chamado, com token válido — Módulo 7).
2. GET http://localhost:8081/api/notificacao/chamado/{id} deve mostrar a notificação criada automaticamente.
3. Pare o Terminal 2 (Ctrl+C) e crie outro chamado. Observe: a criação falha ou demora, mesmo o problema sendo só no serviço de notificação — essa é a prova prática do acoplamento. Suba o Terminal 2 de novo antes de seguir.

Checkpoint

- Criar um chamado gera automaticamente uma notificação no outro serviço.
 - Você reproduziu o cenário de falha e viu o chamados-api ser afetado — essa observação é a motivação real do próximo módulo.
-

Módulo 11 — Comunicação Assíncrona e Desacoplada (RabbitMQ)

Objetivo

Refatorar a comunicação para ser **assíncrona, confiável e com baixo acoplamento** — o requisito final do seu projeto de portfólio.

Teoria: como o RabbitMQ resolve os três requisitos

O RabbitMQ é um **broker de mensagens**: em vez de um serviço chamar o outro diretamente, o publicador manda uma mensagem para o broker, e o broker entrega para quem estiver escutando. O modelo usado é **Exchange → Fila → Consumidor**: o publicador manda a mensagem para um *exchange* (um roteador), o exchange decide para qual *fila* encaminhar (com base numa *routing key*), e o consumidor lê dessa fila.

- **Assíncrona**: publicar uma mensagem não bloqueia — o chamados-api entrega a mensagem ao broker e segue seu fluxo, sem esperar o notificacao-api processar nada.
- **Confiável**: uma fila **durável** sobrevive a reinícios do RabbitMQ, e a mensagem só é removida da fila depois que o consumidor confirma o processamento (ack). Se o consumidor cair no meio do processo, a mensagem some da fila mas o Rabbit também tem mecanismos de *reentrega*; se o processamento falhar continuamente, a mensagem pode ser desviada para uma **Dead Letter Queue (DLQ)** — uma fila separada para mensagens problemáticas, em vez de desaparecerem silenciosamente.
- **Baixo acoplamento**: o chamados-api não sabe (nem precisa saber) que o notificacao-api existe — ele só conhece o formato do evento e o nome da fila/exchange. Um terceiro serviço poderia passar a escutar a mesma fila amanhã sem que uma linha do chamados-api precisasse mudar.

Passo 1 — Adicionar a dependência de mensageria (nos dois projetos)

Pare os dois serviços e adicione, em **cada** pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

mvn clean install em cada projeto antes de continuar.

Passo 2 — Publicador, no chamados-api

```
// techdesk-chamados-
↪ api/src/main/java/com/techdesk/techdesk/chamados/messaging/RabbitMQConfig.java
package com.techdesk.techdesk.chamados.messaging;

import org.springframework.amqp.core.TopicExchange;
import org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.context.annotation.Bean;
```

```

import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitMQConfig {

    public static final String EXCHANGE = "techdesk.exchange";
    public static final String ROUTING_KEY_CHAMADO_CRIADO = "chamado.criado";

    @Bean
    public TopicExchange exchange() {
        return new TopicExchange(EXCHANGE);
    }

    @Bean
    public MessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter(); // converte objetos Java <-> JSON
        ↪ automaticamente
    }
}

```

```

// techdesk-chamados-
↪ api/src/main/java/com/techdesk/techdesk/chamados/messaging/ChamadoCriadoEvent.java
package com.techdesk.techdesk.chamados.messaging;

```

```

import java.time.LocalDateTime;

public record ChamadoCriadoEvent(
    Long chamadoId,
    String titulo,
    String categoriaNome,
    LocalDateTime dataAbertura
) {}

```

```

// techdesk-chamados-
↪ api/src/main/java/com/techdesk/techdesk/chamados/messaging/ChamadoEventPublisher.java
package com.techdesk.techdesk.chamados.messaging;

```

```

import org.springframework.amqp.core.AmqpTemplate;
import org.springframework.stereotype.Component;
import com.techdesk.techdesk.chamados.entity.Chamado;

```

```

@Component
public class ChamadoEventPublisher {

    private final AmqpTemplate amqpTemplate;

    public ChamadoEventPublisher(AmqpTemplate amqpTemplate) {
        this.amqpTemplate = amqpTemplate;
    }

    public void publicarChamadoCriado(Chamado chamado) {
        ChamadoCriadoEvent evento = new ChamadoCriadoEvent(
            chamado.getId(), chamado.getTitulo(),
            chamado.getCategoria().getNome(), chamado.getDataAbertura()
        );
        amqpTemplate.convertAndSend(RabbitMQConfig.EXCHANGE,
        ↪ RabbitMQConfig.ROUTING_KEY_CHAMADO_CRIADO, evento);
    }
}

```

No ChamadoService.criar(...), **troque** a chamada ao NotificacaoClient (Módulo 10) pelo publisher de evento:

```
Chamado salvo = chamadoRepository.save(chamado);
chamadoEventPublisher.publicarChamadoCriado(salvo); // assíncrono – não bloqueia, não
↳ derruba nada
return toResponseDTO(salvo);
```

Passo 3 — Consumidor, no notificacao-api

```
// techdesk-notificacao-
↳ api/src/main/java/com/techdesk/notificacao/notificacoes/messaging/RabbitMQConfig.java
package com.techdesk.notificacao.notificacoes.messaging;

import org.springframework.amqp.core.*;
import org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
import org.springframework.amqp.support.converter.MessageConverter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitMQConfig {

    public static final String EXCHANGE = "techdesk.exchange";
    public static final String FILA_CHAMADO_CRIADO = "notificacao.chamado.criado";
    public static final String ROUTING_KEY = "chamado.criado";
    public static final String DLQ = "notificacao.chamado.criado.dlq";

    @Bean
    public TopicExchange exchange() {
        return new TopicExchange(EXCHANGE);
    }

    @Bean
    public Queue filaChamadoCriado() {
        // Fila DURÁVEL, com Dead Letter Queue configurada: mensagens que falham
        // repetidamente vão para a DLQ, em vez de se perderem.
        return QueueBuilder.durable(FILA_CHAMADO_CRIADO)
            .withArgument("x-dead-letter-exchange", "")
            .withArgument("x-dead-letter-routing-key", DLQ)
            .build();
    }

    @Bean
    public Queue dlq() {
        return QueueBuilder.durable(DLQ).build();
    }

    @Bean
    public Binding binding(Queue filaChamadoCriado, TopicExchange exchange) {
        return BindingBuilder.bind(filaChamadoCriado).to(exchange).with(ROUTING_KEY);
    }

    @Bean
    public MessageConverter jsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }
}
```

```
// techdesk-notificacao-
↪ api/src/main/java/com/techdesk/notificacao/notificacoes/messaging/ChamadoCriadoEvent.java
package com.techdesk.notificacao.notificacoes.messaging;

import java.time.LocalDateTime;

// Mesmo formato do evento do chamados-api – cada serviço tem sua própria
// cópia do "contrato" (JSON), sem compartilhar código Java diretamente.
public record ChamadoCriadoEvent(
    Long chamadoId,
    String titulo,
    String categoriaNome,
    LocalDateTime dataAbertura
) {}

// techdesk-notificacao-
↪ api/src/main/java/com/techdesk/notificacao/notificacoes/messaging/ChamadoEventListener.java
package com.techdesk.notificacao.notificacoes.messaging;

import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Component;
import com.techdesk.notificacao.notificacoes.service.NotificacaoService;

@Component
public class ChamadoEventListener {

    private final NotificacaoService notificacaoService;

    public ChamadoEventListener(NotificacaoService notificacaoService) {
        this.notificacaoService = notificacaoService;
    }

    @RabbitListener(queues = RabbitMQConfig.FILA_CHAMADO_CRIADO)
    public void ouvirChamadoCriado(ChamadoCriadoEvent evento) {
        notificacaoService.registrar(
            evento.chamadoId(), evento.titulo(),
            "Novo chamado aberto: " + evento.titulo() + " (categoria: " +
            ↪ evento.categoriaNome() + ")"
        );
        // Terminar sem exceção = ack automático (mensagem sai da fila).
        // Lançar exceção = reentrega e, após tentativas, envio à DLQ.
    }
}

# adicionar em application.yaml dos DOIS serviços
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: guest
    password: guest
```

Teste no Postman + painel do RabbitMQ

1. Suba os dois serviços de novo.
2. Crie um chamado pelo Postman.
3. Confira a notificação em GET /api/notificacao/chamado/{id} — agora criada de forma assíncrona.
4. Pare o Terminal 2 e crie outro chamado: dessa vez a criação funciona normalmente (sem

demora). Suba o Terminal 2 de novo — a mensagem pendente é processada sem nenhuma ação manual sua.

5. Acesse `http://localhost:15672` → aba **Queues** → veja a fila `notificacao.chamado.criado` e observe mensagens acumulando enquanto o consumidor está fora do ar.

Checkpoint

- Com o `notificacao-api` fora do ar, criar chamado continua funcionando normalmente — prova de que o acoplamento foi removido.
- Ao religar o serviço, as mensagens pendentes são processadas sozinhas.
- Você consegue explicar, numa frase cada, o que torna essa comunicação assíncrona, confiável e de baixo acoplamento.

Módulo 12 — Frontend Consumindo a API

Objetivo

Criar uma tela simples para abrir e listar chamados — o rosto visual do projeto.

Teoria: CORS

Por padrão, o navegador bloqueia um `fetch` de uma origem para outra (ex: `localhost:5500` chamando `localhost:8080`). Resolvemos com uma configuração de CORS explícita no backend.

```
// techdesk-chamados-api/src/main/java/com/techdesk/techdesk/config/CorsConfig.java
package com.techdesk.techdesk.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/api/**")
                    .allowedOrigins("http://localhost:5500", "http://127.0.0.1:5500")
                    .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE")
                    .allowedHeaders("*");
            }
        };
    }
}
```

```
<!-- techdesk/frontend/index.html -->
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <title>TechDesk – Painel de Chamados</title>
    <link rel="stylesheet" href="style.css">
</head>
```

```

<body>
  <h1>TechDesk</h1>
  <form id="form-chamado">
    <input type="text" id="titulo" placeholder="Título" required>
    <textarea id="descricao" placeholder="Descrição" required></textarea>
    <input type="number" id="categoriaId" placeholder="ID da categoria" required>
    <button type="submit">Abrir chamado</button>
  </form>
  <table id="tabela-chamados">
    <thead><tr><th>Título</th><th>Status</th><th>Categoria</th></tr></thead>
    <tbody></tbody>
  </table>
  <script src="app.js"></script>
</body>
</html>

```

```

// techdesk/frontend/app.js
const API_URL = "http://localhost:8080/api/chamado"; // singular – igual ao seu backend
↪ real
const token = localStorage.getItem("token");

async function carregarChamados() {
  const resposta = await fetch(API_URL, { headers: { "Authorization": `Bearer ${token}`
  ↪ } });
  const chamados = await resposta.json();
  const corpo = document.querySelector("#tabela-chamados tbody");
  corpo.innerHTML = "";
  chamados.forEach(c => {
    const linha = document.createElement("tr");
    linha.innerHTML =
    ↪ `<td>${c.titulo}</td><td>${c.status}</td><td>${c.categoriaNome}</td>`;
    corpo.appendChild(linha);
  });
}

document.getElementById("form-chamado").addEventListener("submit", async (e) => {
  e.preventDefault();
  const novo = {
    titulo: document.getElementById("titulo").value,
    descricao: document.getElementById("descricao").value,
    categoriaId: Number(document.getElementById("categoriaId").value)
  };
  await fetch(API_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json", "Authorization": `Bearer ${token}`
    ↪ },
    body: JSON.stringify(novo)
  });
  document.getElementById("form-chamado").reset();
  carregarChamados();
});

carregarChamados();

```

Conseguir um token (o seu login usa login/senha, não e-mail)

No Postman: POST `http://localhost:8080/api/auth/login`, body `{"login": "elimar", "senha": "senha123"}`. Copie o token da resposta. No navegador, com o frontend aberto, abra o Console (F12) e rode:

```
localStorage.setItem("token", "COLE_O_TOKEN_AQUI");
```

Lembre-se: o token expira em 1 minuto na configuração atual (Módulo 8) — se demorar, faça login de novo.

Servir o frontend

```
cd techdesk/frontend  
python3 -m http.server 5500
```

Acesse `http://localhost:5500` — nunca `file://`.

Checkpoint

- A tabela carrega os chamados sem erro de CORS no console do navegador.
- Criar um chamado pelo formulário atualiza a tabela automaticamente.

Módulo 13 — Docker Compose Multi-Serviço (Aprofundamento)

Objetivo

Empacotar os dois serviços, os três bancos e o RabbitMQ num único `docker-compose.yml`.

Passo 1 — Encerrar os containers manuais

```
cd techdesk  
docker-compose down
```

Feche também os Terminais 1 e 2.

Passo 2 — Dockerfile de cada serviço

```
# techdesk-chamados-api/Dockerfile  
FROM eclipse-temurin:21-jdk-alpine AS build  
WORKDIR /app  
COPY . .  
RUN ./mvnw clean package -DskipTests  
  
FROM eclipse-temurin:21-jre-alpine  
WORKDIR /app  
COPY --from=build /app/target/*.jar app.jar  
EXPOSE 8080  
ENTRYPOINT ["java", "-jar", "app.jar"]
```

(Repita para `techdesk-notificacao-api/Dockerfile`, trocando `EXPOSE` para 8081.)

Passo 3 — docker-compose.yml final

```
# techdesk/docker-compose.yml (versão final)  
version: '3.8'  
  
services:  
  mysql-chamados:
```

```

image: mysql:8.0
environment:
  MYSQL_ROOT_PASSWORD: techdesk123
  MYSQL_DATABASE: techdesk_chamados
ports: ["3307:3306"]
volumes: ["mysql-chamados-data:/var/lib/mysql"]

mysql-notificacao:
image: mysql:8.0
environment:
  MYSQL_ROOT_PASSWORD: techdesk123
  MYSQL_DATABASE: techdesk_notificacao
ports: ["3308:3306"]
volumes: ["mysql-notificacao-data:/var/lib/mysql"]

rabbitmq:
image: rabbitmq:3-management
ports: ["5672:5672", "15672:15672"]

chamados-api:
build: ./techdesk-chamados-api
depends_on: [mysql-chamados, rabbitmq]
environment:
  SPRING_DATASOURCE_URL: jdbc:mysql://mysql-chamados:3306/techdesk_chamados
  SPRING_RABBITMQ_HOST: rabbitmq
ports: ["8080:8080"]

notificacao-api:
build: ./techdesk-notificacao-api
depends_on: [mysql-notificacao, rabbitmq]
environment:
  SPRING_DATASOURCE_URL: jdbc:mysql://mysql-notificacao:3306/techdesk_notificacao
  SPRING_RABBITMQ_HOST: rabbitmq
ports: ["8081:8081"]

volumes:
  mysql-chamados-data:
  mysql-notificacao-data:

```

Note a diferença de porta: **dentro** da rede Docker, todo mundo fala com o MySQL pela porta interna 3306 (o nome do serviço, tipo mysql-chamados, faz o papel de “endereço”). As portas 3307/3308 só importam para você acessar o banco **de fora**, do seu próprio computador.

```
docker-compose up --build
```

Checkpoint

- Um único docker-compose up --build sobe os 5 containers e o sistema funciona de ponta a ponta.
- A collection do Postman continua passando nos mesmos testes de antes, agora contra os containers.

Módulo 14 — Publicação, GitHub e Portfólio

Objetivo

Deixar o projeto pronto para ser avaliado por um recrutador ou entrevistador técnico.

O que destacar (você já tem mais do que o básico)

Ao escrever o README e ao falar do projeto em entrevista, não venda só “fiz um CRUD” — você tem bem mais que isso: - Criação e exclusão em lote (/api/chamado/lote, DELETE /api/chamado), não só CRUD unitário. - Atualização parcial via PATCH, com validação de status (StatusChamadoNaoExisteException). - Regra de negócio real protegendo integridade de dados: categoria não pode ser excluída se tiver chamados associados. - Autenticação JWT com papéis (ADMIN/USER) e rotas administrativas segregadas. - Arquitetura de dois microsserviços se comunicando de forma assíncrona e desacoplada via RabbitMQ, com Dead Letter Queue. - Testes automatizados cobrindo regras de negócio, não só “o feliz caminho”.

Passo a passo

1. Adicione Swagger/OpenAPI (springdoc-openapi-starter-webmvc-ui) no chamados-api, em /swagger-ui.html.
2. README.md na raiz cobrindo: o que o sistema faz, arquitetura (diagrama simples dos dois serviços + RabbitMQ + bancos), tecnologias, como rodar (docker-compose up), prints do frontend, link da collection do Postman.
3. Confirme o histórico de commits organizado no GitHub (você já tem um bom histórico incremental — mantenha o padrão).
4. No currículo/LinkedIn: *“TechDesk — sistema de gestão de chamados técnicos com arquitetura de microsserviços em Java 21/Spring Boot 4, comunicação assíncrona via RabbitMQ com DLQ, autenticação JWT com controle de papéis, testes automatizados (JUnit/Mockito) e frontend integrado, containerizado com Docker.”*

Checkpoint final

- Alguém de fora, só lendo o README e rodando docker-compose up, consegue subir o projeto sem te perguntar nada.
- Você consegue explicar em até 2 minutos a arquitetura completa e por que a comunicação entre os serviços é assíncrona — essa é a pergunta que um entrevistador júnior/pleno faz ao ver esse projeto no currículo.